ROUND 2 SUBMISSION / BASELINE 22a05ef

INKSHIELD

Evidence-first brand governance - verify before you publish.

CHECK / FIND EVIDENCE / EXPLAIN / APPROVE

A pre-publication governance step between draft and publish:
deterministic scanning, evidence-backed findings, human decisions, audit trail.

NEXT.JS 16 + FASTAPI / VERCEL + RENDER / 48 FRONTEND + 52 BACKEND TESTS PASSING

What it is. INKSHIELD checks draft content against approved brand knowledge, returns findings that always cite the knowledge record behind them, and records every scan and human decision in an audit trail.

LIVE FRONTEND	https://inkshield-rho.vercel.app
LIVE API	https://inkshield-api.onrender.com
REPOSITORY	https://github.com/Akash-Selvaraj-R/INKSHIELD
BASELINE	22a05ef on main (verified 2026-10-29)
SOURCE OF THIS PDF	docs/INKSHIELD-Round2-Submission.md

IN THIS DOCUMENT

01 Problem and target users
02 Solution and end-to-end workflow
03 Architecture and technology choices
04 Completed features and testing evidence
05 Progress and changes since Round 1
06 Known limitations
07 Three concrete Round 3 improvements
08 How to verify this submission

1. Problem and target users

The problem

Brand, product-marketing, and compliance teams publish content faster than anyone can verify it. The result is predictable: performance claims drift away from the approved messaging, prohibited phrases such as "100% uptime" appear in ads, product names get written the old way — and when an incorrect claim ships, nobody can say which knowledge source it contradicted or who approved it.

Existing options fail these teams in two directions:

- **Manual legal/brand review** is accurate but slow, inconsistent, and impossible to audit after the fact.
- **Generic "AI content checkers"** produce opaque scores and unverifiable claims, training users to click through warnings.

What is missing is a governance step between "draft" and "publish" that is *fast enough to always run*, *transparent enough to trust*, and *accountable enough to audit*.

Target users

- **Content and product-marketing authors** who need an immediate pre-publication check of claims, terminology, and product facts.
- **Brand and compliance reviewers** who need evidence rather than vibes: every finding must cite the exact knowledge record (source and section) that it violates.
- **Team leads and auditors** who need a durable record of what was scanned, what was decided, and the state transition of every decision.

Design principle

> ***AI makes a recommendation. Evidence supports the recommendation. A human owns the decision.***

INKSHIELD never auto-blocks or silently "fixes" content. It produces findings with evidence, and a person decides — approve, reject, edit, or escalate.

2. Solution and end-to-end workflow

INKSHIELD is a two-service web application: a Next.js frontend where authors and reviewers work, and a FastAPI backend that performs scanning, review, and audit duties. The complete workflow runs end to end on the live deployment.

The workflow, step by step

1. **Submit** — the author pastes draft content on `/scan`, chooses a content type, and selects which of the five knowledge sources to check against (or all of them).
2. **Validate** — the API rejects empty content and invalid input with HTTP 422 and a precise message; nothing runs on invalid input.
3. **Select knowledge** — the backend resolves the requested knowledge sources (`claims`, `terms`, `guidelines`, `prohibited`, `facts`). Unknown ids are ignored, never fabricated; an empty selection means the complete knowledge base.

4. **Scan (deterministic rule engine)** — four rule families run in priority order: quantitative claims, prohibited absolutes, superlatives, and terminology. Overlapping matches are suppressed and findings are numbered sequentially in reading order.
5. **Report with evidence** — each finding carries severity, submitted text, suggested correction, explanation, confidence, and at least one evidence record that quotes a real seeded knowledge source (`source`, `section`, `text`). A compliance score starts at 100 and drops by fixed weights (CRITICAL 15, HIGH 10, MEDIUM 5, LOW 2).
6. **Human review** — a reviewer opens a finding and chooses Approve, Reject, Edit & approve (corrected text required), or Escalate. The backend validates the decision, updates only that finding, and never alters the deterministic scan data (title, explanation, evidence, confidence, score).
7. **Audit and persistence** — the scan and the decision each append an audit event with actor, action, and previous -> new state. The updated finding and the event remain readable through `GET /api/v1/scans` and `GET /api/v1/audit-events`, so state survives a page reload.

Example scan response (real, abbreviated from the API)

```
{
  "id": "scan_1a2b3c4d5e6f",
  "status": "completed",
  "compliance_score": 45,
  "summary": { "total_findings": 2, "critical": 1, "high": 1, "medium": 0, "low": 0 },
  "findings": [
    {
      "id": "INK-0042",
      "category": "CLAIM",
      "severity": "CRITICAL",
      "status": "OPEN",
      "title": "Unsupported performance claim",
      "submitted_text": "50% faster inference",
"explanation": "The submitted performance claim exceeds the maximum approved quantitative claim
      in the brand knowledge base (up to 30% faster).",
      "suggested_correction": "Up to 30% faster inference.",
      "confidence": 96,
"evidence": [{ "source": "Product Claims", "section": "§4.2", "text": "Up to 30% faster
      inference." }],
      "review": { "required": true, "decision": null },
      "corrected_text": null
    }
  ]
}
```

Failure behavior (part of the product)

- Unreachable backend or non-2xx response -> the UI shows a precise error message; no finding is displayed, no state is changed optimistically.
- Failed requests (404/422) -> no audit event is ever recorded, so the audit trail only contains things that really happened.
- Production build without `NEXT_PUBLIC_API_URL` -> scans fail with an explicit configuration error instead of silently calling localhost.

3. Architecture and technology choices

System architecture

```
Browser
  |  Next.js 16 App Router (app/, app/app-shell.tsx)
  |  typed client: lib/api/client.ts  (base URL: NEXT_PUBLIC_API_URL)
  v
FastAPI backend (backend/, base path /api/v1)
  routes   ->  services  ->  rules / knowledge
   |            | scanner.py    scan pipeline + in-memory ScanStore
   |            | audit.py      in-memory append-only AuditLog
   |            | knowledge.py  seeded NOVA knowledge sources
   |            v
   |        Pydantic v2 request/response validation
   v
Process memory only (no database; reset on restart)

Deployment: Vercel (frontend) <-> Render web service (backend,
render.yaml blueprint, health check /api/v1/health)
```

Layering rule: `routes -> services -> rules/knowledge`, with `schemas` and `core/config` at the edges. Policy content (limits, prohibited phrases, approved terms) lives only in `services/knowledge.py`; the rule engine never hardcodes brand-specific strings, so new knowledge sources do not require engine changes.

API surface (versioned, JSON)

Method	Endpoint	Purpose
GET	/api/v1/health	Health and API version
POST	/api/v1/scans	Scan content; returns findings, summary, score (201)
GET	/api/v1/scans	Scans kept for the current server process
PATCH	/api/v1/scans/{id}/findings/{id}/review	Record a human decision (200)
GET	/api/v1/audit-events	Audit events in chronological order

Technology choices and rationale

Choice	Rationale
FastAPI + Pydantic v2	Typed request/response contract with automatic validation (422 with precise detail) and interactive OpenAPI docs at /docs.
Deterministic rule engine	Reproducible, testable, and evidence-resolvable by construction: the same input always yields the same findings, so results can be asserted in tests. No API keys, no cost per scan, no hallucinated citations.
Next.js 16 + React 19 + strict TypeScript	Server-rendered React app with one typed API client (`lib/api/client.ts`); snake_case API responses are mapped once instead of being scattered through components.
Tailwind CSS 4 + shadcn/base-ui	Fast iteration on a distinctive, accessible interface without a heavyweight component system.
Vitest + pytest (TestClient)	Two fast, dependency-free test suites: pure frontend logic and full HTTP API behavior, no running server required.
Vercel + Render free tier	Zero-cost production deployment for both services; `render.yaml` keeps the backend service definition in the repository.

Choice	Rationale
In-memory storage (deliberate v1 scope)	Zero operational setup for the demo while the workflow was being proven; persistence is the first Round 3 item (see section 6).

Technology stack (versions as installed)

Layer	Stack
Frontend	Next.js 16.4.3 (App Router, Turbopack), React 19.2, TypeScript 5.7.3 (strict), Tailwind CSS 4.3, lucide-react, Vercel Analytics
Backend	Python 3.11+ (developed and tested on 3.14), FastAPI 0.162, Pydantic 2.14, uvicorn
Testing	Vitest 5.0 (frontend), pytest 9.2 + HTTPX 0.28 (backend)
Deployment	Vercel (frontend), Render (backend, Python runtime)
Storage	In-process memory (thread-safe lists guarded by locks); no database

4. Completed features and testing evidence

Completed features (all verified in the repository)

- Deterministic knowledge-driven scanner with four rule families, overlap suppression, sequential finding IDs from INK-0042, and reading-order output.
- Evidence-first findings: every finding cites real seeded knowledge records (`source`, `section`, `text`); a dedicated test enforces the "no fabricated evidence" invariant.
- Compliance scoring (100 minus fixed severity weights, clamped to [0,100]) plus severity summary.
- Selectable knowledge sources with safe handling of unknown ids (ignored, never fabricated).
- Full HTTP API: health, create scan, list scans, record review decision, list audit events; strict input validation with precise 422/404 errors.
- Human review workflow with four decisions (APPROVED, REJECTED, EDITED, ESCALATED); `EDITED` requires corrected text; deterministic scan data is never mutated by a review; decisions persist across reloads.
- Append-only audit log: successful scans and decisions append events with actor type (SYSTEM/HUMAN), action, and previous -> new state; failed requests append nothing.
- Typed frontend API client with snake_case/camelCase interop, error extraction from FastAPI 422/404 bodies, and a production guard that refuses to run without `NEXT_PUBLIC_API_URL`.
- Evidence-first UI: scan page with source selection, findings with evidence cards and highlighted content, review drawer with the four actions, live audit trail page, and explicit error states (no optimistic updates).
- Deployment configuration: `render.yaml` blueprint, CORS origin configuration, Vercel environment variable documented.

Automated test evidence (run 2026-10-29, baseline 22a05ef)

Check	Command	Result
Frontend unit tests	pnpm test (Vitest)	48/48 passed, 4 test files

Check	Command	Result
Backend API tests	python -m pytest backend/tests -q	52/52 passed
TypeScript strict check	tsc --noEmit	0 errors
Production build	pnpm build	Succeeded (Next.js 16.4.3, Turbopack)
Live API health	GET https://inkshield-api.onrender.com/api/v1/health	HTTP 200 healthy
Live frontend	GET https://inkshield-rho.vercel.app	HTTP 200

Test coverage by area:

- **API client (11 tests):** create scan request/response mapping, 422 detail extraction, unreachable backend; review decision mapping (EDITED vs APPROVED), 404 detail, unreachable backend; audit event mapping, empty list, 503, unreachable backend.
- **Rule engine (13 tests):** all four rule families, sequential IDs from INK-0042, overlap suppression, superlative handling, demo draft.
- **Scan pipeline (7 tests):** scoring, summarization, determinism, empty-content rejection, full demo pipeline.
- **Knowledge base (17 tests):** source selection, knowledge-selection scenarios, the no-fabricated-evidence invariant, rule compilation, API-shaped knowledge copies.
- **Backend API (52 tests):** health (2), scans (23: validation, stable scores, reading order, determinism, rule families, evidence resolution, knowledge filtering, listing), reviews (11: all decisions, casing variants, 404/422, persistence, deterministic-data preservation, CORS preflight), audit (12: events per action, no events on failures, chronological order, persistence), CORS (4).

Manual verification of the live workflow

The complete workflow was manually verified against the production deployment:

- Risky copy scanning returns real findings with evidence (source and section) for claims, prohibited absolutes, superlatives, and terminology.
- Human review actions (approve, reject, edit & approve, escalate) succeed and update only the reviewed finding.
- Review state persists after a page reload (read back from `GET /api/v1/scans`).
- Audit events appear for scans and decisions with correct state transitions.
- Error handling produces honest errors: no fabricated findings, no fabricated audit events, failed requests recorded nowhere.

5. Progress and changes since Round 1

The repository does not archive the Round 1 submission, so this section reports progress strictly from the project's git history: eight commits on `main`, from the first frontend commit to the deployment-ready baseline `22a05ef`.

Commit	Date	Change
9ef59ee	2026-10-27	Initialize the INKSHIELD frontend (landing page, workspace shell, visual system)
470ee13	2026-10-27	Add the deterministic scanner and knowledge-driven rules (TypeScript scanner, rule engine, seeded knowledge base, first test suites)
15d91ad	2026-10-28	Add the FastAPI backend foundation (app factory, versioned router, health endpoint, Pydantic schemas)
2ebedf6	2026-10-28	Connect the frontend to the FastAPI scanner (typed client, live scan flow replacing static-only results)
14912a3	2026-10-28	Add the human review workflow (four decisions, validation, persistence)
527681e	2026-10-28	Stop tracking the tsconfig.tsbuildinfo build artifact
51f2778	2026-10-28	Add the audit event architecture (append-only log, events per scan/decision, audit API and UI)
22a05ef	2026-10-28	Prepare the backend for Render deployment (render.yaml blueprint, CORS/production configuration, deployment docs)

What this means for judges:

- **A prototype became a full-stack system:** the frontend-only prototype gained a real API service with its own contract, tests, and deployment target.
- **The core governance loop is now end to end:** scan -> findings with evidence -> human decision -> persisted state -> audit event, all through the HTTP API rather than static demo objects.
- **Accountability was added:** audit events record what actually happened (system scans and human decisions) and never record failures.
- **It ships:** production deployments exist for both services, with health checks, CORS configuration, and reproducible setup documented in the repository.

6. Known limitations

These are real and deliberately stated; none are hidden behind demo data.

1. **In-memory scan and audit storage.** Scans, review decisions, and audit events live in process memory only — there is no database. Every restart, redeploy, or free-tier spin-down clears them. Endpoints keep working afterwards, but history starts empty.
2. **Cold starts on the free backend.** The Render free-tier service sleeps when idle; the first request after idle can take several seconds while the instance wakes (and a request may time out and need a retry).
3. **No authentication.** There are no user accounts, so audit events attribute human decisions to a generic "Reviewer" role.
4. **Deterministic scanner, not a language model.** Scanning is rule-based against a seeded demo knowledge base (five sources for the fictional NOVA brand). There is no RAG or external LLM call, so coverage is exactly what the rules encode.
5. **Duplicated scanner logic.** The Python scanner is an independent port of the TypeScript scanner; the two are kept equivalent by separate test suites rather than shared code or a shared schema.
6. **Partially wired UI.** The Dashboard KPIs, Knowledge Base counts, and Review Queue items are illustrative demo content, and Scan History / Settings are labeled placeholders; the fully live path is Scan

Content -> results -> review -> Audit Trail.

7. **Build-time type errors ignored by Next.** `next.config.mjs` sets `typescript.ignoreBuildErrors`, so `tsc --noEmit` is the authoritative type check; there is no lint script configured yet.

7. Three concrete Round 3 improvements

1. Durable storage for scans, reviews, and audit events

What: Introduce a storage layer (SQLite for development, PostgreSQL on Render) behind the existing `ScanStore` and `AuditLog` interfaces, with SQLAlchemy models and migrations, so scans, findings, review decisions, and audit events survive restarts.

Rationale: This removes the single biggest limitation (1) and is a prerequisite for every other accountability feature. The audit trail is only meaningful if it outlives the process that wrote it, and judges re-running the demo after a redeploy should see earlier activity rather than an empty trail. Because both stores are already isolated behind small interfaces and all 52 backend tests exercise them through the HTTP API, the swap is testable without redesigning the product.

Verification: existing pytest suite runs against the database; a new test restarts the app factory and asserts scans and audit events persist.

2. Reviewer identity, authentication, and assignment

What: Add lightweight authentication (session or token based), attribute every audit event to the authenticated actor instead of the generic "Reviewer", enforce that only authorized users can record decisions, and wire the Review Queue page to real pending findings with assignment.

Rationale: Human-in-the-loop governance without identity only claims accountability; with it, the audit trail answers *who* approved a claim, and review queues gain real meaning (the UI already shows "Unassigned / Assigned to me" as static text). Limitation (3) disappears, and the same mechanism provides a natural place to gate knowledge-base administration later.

Verification: audit events carry distinct actors; unauthenticated PATCH requests are rejected; a new test asserts actor attribution end to end.

3. Grounded knowledge retrieval (RAG) with citation enforcement

What: Let teams ingest their own brand documents (Markdown/PDF) into the knowledge service, and add an LLM-assisted analysis pass that may only return a finding if it resolves to a real retrieved knowledge record; keep the deterministic rule engine as the fast, always-on first pass. Findings that cannot cite a resolvable source are discarded by the API — never surfaced.

Rationale: Rule families cover four risk patterns well, but real brand knowledge is large and evolving; retrieval extends coverage to paraphrased claims and context-dependent wording while preserving the product's core promise — every recommendation has a source. Citation enforcement keeps the no-fabrication invariant enforceable in tests, and the deterministic layer keeps scans reproducible and free of per-scan cost when an LLM is unavailable.

Verification: the existing 48 + 52 tests must still pass unchanged (determinism and evidence invariants), plus new tests asserting that every AI-assisted finding carries evidence that resolves against stored knowledge.

8. How to verify this submission

1. Open https://inkshield-rho.vercel.app and follow the demo in the repository README (Scan Content -> review -> Audit Trail).
2. Check the API directly: `GET https://inkshield-api.onrender.com/api/v1/health` and browse `https://inkshield-api.onrender.com/docs`.
3. Clone https://github.com/Akash-Selvaraj-R/INKSHIELD at baseline `22a05ef` or newer and run `pnpm test`, `python -m pytest backend/tests -q`, `tsc --noEmit`, and `pnpm build`.
4. Read `backend/README.md` for the full API contract and `README.md` for setup, walkthrough, and limitations.

All numbers in this document come from those commands and the repository itself; no user counts, benchmarks, customers, or judging results are claimed anywhere.